

CORE LOGIC

◆ Step 1: Data Write

```
HADDR = 0x00
HWRITE = 1
→ fifo_wr_en = 1
→ HWDATA[7:0] stored in FIFO
→ wptr++
```



◆ Step 2: FIFO Status Update

```
wptr != rptr
→ fifo_empty = 0
```



◆ Step 3: FSM Detects Data

```
State = IDLE
if (fifo_empty == 0)
→ state = POP
```



◆ Step 4: POP State (Read FIFO)

```
fifo_rd_en = 1
→ FIFO outputs data on fifo_dout
→ rptr++
```



◆ Step 5: LAUNCH State

```
tx_start = 1
```



fifo_dout → directly connected to tx_data

◆ Step 6: SPI Master Starts

Condition:

```
if (tx_start && !active)
```



Means: Data available, No ongoing transmission

◆ Step 7: Load into Shift Register

```
shreg <= tx_data  
spi_cs = 0  
active = 1
```



◆ Step 8: Serial Transmission

On every `clk_en`:

```
spi_sclk toggles  
→ MOSI outputs MSB  
→ shift register shifts
```



◆ Step 9: Completion

```
bitcnt == 0  
→ tx_done = 1  
→ spi_cs = 1  
→ active = 0
```



◆ Step 10: FSM Goes Back

```
WAIT → IDLE  
→ checks fifo again
```



CLOCK LOGIC CORE

◆ Step 1: User sets divider

```
HADDR = 0x1C  
HWRITE = 1  
→ clk_div <= HWDATA[25:0]
```



◆ Step 2: Counter starts running

```
On every system clock (100 MHz):  
div_cnt++
```



◆ Step 3: Compare with divider

```
if (div_cnt == clk_div)
```



When match happens

```
clk_en = 1  
div_cnt = 0
```



◆ Step 4: Enable pulse triggers toggle

```
if (clk_en)  
    spi_sclk <= ~spi_sclk
```



◆ Step 5: Clock toggles

```
0 → 1 → 0 → 1 → ...
```



◆ Step 6: Full SPI clock cycle

```
2 toggles = 1 full clock cycle
```



◆ Step 7: Final Frequency

$$f_{SPI} = \frac{100 \text{ MHz}}{2 \times (\text{clk_div} + 1)}$$

TRANSLATION CORE

◆ Step 1: CPU sends AXI write request

```
AWADDR → address  
WDATA  → data  
AWVALID = 1  
WVALID  = 1
```



CPU says: “Write this DATA to this ADDRESS”

◆ Step 2: AXI handshake happens

```
if (AWVALID && AWREADY) → address accepted  
if (WVALID && WREADY)  → data accepted
```



Bridge behavior: AWREADY = 1 → accept address, WREADY = 1 → accept data

◆ Step 3: Store in buffers

```
awbuf ← AWADDR  
wbuf  ← WDATA  
aw_pending = 1  
w_pending  = 1
```



AXI gives address & data separately → so we store them

◆ Step 4: Check both available

```
if (aw_pending && w_pending)
```



Now bridge is ready to convert

◆ Step 5: Generate AHB signals

```
HADDR  = awbuf  
HWDATA = wbuf  
HWRITE = 1  
HTRANS = NONSEQ
```



Meaning: “Perform a single AHB write transaction”

◆ Step 6: Wait for AHB ready

```
if (HREADY == 1)
```



Transaction complete

◆ Step 7: Send response back to CPU

```
BVALID = 1  
BRESP = OKAY
```



CPU acknowledges: if (BREADY) → back to IDLE